Event_Data_Stream_Simulation_v3.m: Documentation

Last updated October 21, 2026. Companion documents: Event_Data_Diagnostics.m Documentation and Event Simulator Calibration Plan.

1. Purpose

This script simulates what an event camera records when one bright point source (a satellite or piece of debris) moves in a straight line across the sensor. It implements the stochastic pixel model from Benson and Holzinger (2022), Equations 1-15, and writes the events to a text file laid out like a Prophesee export (x, y, polarity, time). That file can be fed to an angular-rate or tracking pipeline. The script also prints the ground-truth angular rate, so the pipeline's answer can be checked against the known one.

2. Quick start

1. Open the script in MATLAB (R2019a or newer recommended; see section 9).
2. Set the sensor size `nx`, `ny` to match your pipeline and choose `bgMode` ('clean' or 'full noise').
3. Run it. Progress prints every 10% of pixel columns.
4. Outputs: `synthetic_debris_events.txt` in the current folder, a console summary with ground truth, and two figures.
5. For calibration statistics and plots, run Event_Data_Diagnostics.m on the output file.

3. The pixel model in plain language

Every pixel is a leaky bucket that is simulated independently of all other pixels.

- Each photon that lands on the pixel adds a jump to the pixel voltage V. The jump size is `h = b * (ln(lambda) - ln(lambda_r)) / lambda` (Eq. 3), where lambda is the pixel's photon rate at that moment.
- Between photons, V leaks away: `dV/dt = -a * V`.

- The pixel remembers a reference voltage V_ref. If `V - V_ref` reaches `c_pos` it fires a +1 event. If it reaches `c_neg` it fires a -1 event. After either event, V_ref is reset to the current V.
- The thresholds are `c_pos = (b/a) * ln(1 + Delta_lam)` and `c_neg = (b/a) * ln(1 - Delta_lam)`, which is negative (Eq. 5).
- For steady light the average voltage settles at `V = (b/a) * ln(lambda / lambda_r)` (Eq. 9).
- The light on pixel (i, j) is `lambda_ij(t) = lam_bg + Phi(t) * Fx * Fy` (Eq. 15). Fx and Fy are the fractions of a 2-D Gaussian blur (the PSF) that fall inside the pixel in x and y, computed with the error function. `Phi(t) = Phi_0 * (1 + mod_depth * sin(2*pi*f_spin*t))`.

What this produces: as the target arrives, a pixel's voltage ramps up and fires + events near the target's centroid (the bow shock). After the target leaves, the voltage decays and fires - events, so a tail of negative events trails behind. The tail length scales roughly with v/a (speed divided by decay rate).

4. Background modes

Mode	lambda_r	Pixels simulated	Where noise comes from	Speed
'clean'	equal to lam_bg	only a box around the track	random events added on top (`noiseRate`)	fast
'full noise'	`lambda_r_full` (below lam_bg)	every pixel on the sensor	sky photons passing through the circuit	slow

Why clean mode is silent: when lambda_r equals lam_bg, a sky photon has `h = b * (ln(lam_bg) - ln(lam_bg)) / lam_bg = 0`, so it adds no voltage and cannot cause an event. Pixels away from the target therefore produce nothing and can be skipped.

In full noise mode a lower lambda_r makes every sky photon a big jump, followed by a run of decay events. For large rates the background is approximately `a * ln(lam_bg / lambda_r) / |ln(1 - Delta_lam)|` events/s/pixel. This formula fails below roughly 10 events/s because the jumps become too small to cross a threshold, so tune lambda_r_full from a sweep instead. Measured values with a = 10 and lam_bg = 5 (Python port of the pixel logic, 100 pixel-runs of 20 s):

lambda_r_full	Delta_lam = 0.10 (events/s/pixel)	Delta_lam = 0.25 (events/s/pixel)
4.00	23.9	9.0
4.25	18.1	5.0
4.50	12.4	1.3
4.60	8.3	0.56
4.70	4.5	0.22
4.80	1.1	0.03
4.90	0.11	0.00
4.95	0.00	0.00

At low rates the + and - fractions come out close to 50/50. At high rates the noise is mostly - events.

5. Input reference

Run settings

Variable	Default	Units	What it does
`seed`	1	none	Random seed. The same seed reproduces the same output. Use 'shuffle' for a new realization each run.
`bgMode`	'clean'	none	'clean' or 'full noise' (section 4). Any other string is an error.
`nx` , `ny`	1280, 720	px	Sensor size. Must match the pipeline that reads the file.
`T_sim`	2.09	s	Total simulated time.
`dt`	1e-3	s	Spacing of the samples used to look up the target's light on each pixel. It is not the photon time resolution. Keep target motion per step (speed times dt) below about 0.3 px.

Variable	Default	Units	What it does
`polarity01`	true	none	true writes polarity as 0/1 (Prophesee style); false writes -1/+1.

Noise inputs

Variable	Default	Units	Used in	What it does
`noiseRate`	0.10	events/s/pixel	clean	Random events added on top. Count is about noiseRate x nx x ny x T_sim, with uniform position and time and 50/50 polarity.
`lambda_r_full`	4.8	photons/s	full noise	Reference flux. Closer to lam_bg is quieter; equal to lam_bg gives zero noise.
`maxEventsAllowed`	3e7	events	full noise	Safety cap. The script stops with an error if its event estimate exceeds this.
`lam_bg`	5	photons/s/pixel	both	Sky background flux on every pixel.

Event circuit

Variable	Default	Units	What it does
`a`	10	1/s	Voltage decay rate. Larger gives a shorter tail and, in full noise mode, more background events.
`b`	100	none	Gain. It scales the jumps and the thresholds equally, so it does not change event rates for steady light.
`Delta_lam`	0.10	fraction	Relative brightness change needed for one event. Larger gives fewer events. The paper quotes about 0.25 as the EVK3 default.

Target and optics

Variable	Default	Units	What it does
`x0` , `y0`	1280, 90	px	Starting centroid of the target.
`vx` , `vy`	-612.44, 43.06	px/s	Velocity. Image y points down, so positive vy moves the target down the screen.
`Phi_0`	12000	photons/s	Average total target flux, summed over all pixels its blur covers.
`mod_depth`	0.0	fraction	Flicker depth. 0 is steady, 1 fades to zero.
`f_spin`	10	Hz	Flicker frequency. Ignored when mod_depth is 0.
`sigma_psf`	1.5	px	Gaussian blur standard deviation. FWHM is about 2.355 times this.
`pixPitch_m` , `focalLen_m`	4.86e-6, 0.6	m	Used only to print the ground-truth angular rate. They give an IFOV of about 1.671 arcsec/px.

6. Code walkthrough

Section	What happens
0. Random seed	Calls `rng(seed)`.
1-4. Settings	The user inputs above.
5. Derived quantities	Checks bgMode, sets lambda_r (lam_bg in clean mode, lambda_r_full in full noise), computes c_pos and c_neg, and bundles the circuit constants in the struct `p`.
6. Target motion	Builds the time grid, the centroid path x_bar(t), y_bar(t) and the brightness Phi_t. Warns if the track is within m = ceil(5 sigma) + 1 pixels of a sensor edge.
7. Pixel selection	Clean mode simulates only the bounding box of the track plus margin m. Full noise mode simulates every pixel and first estimates the event count against `maxEventsAllowed`.
8. Pixel loop	Precomputes Fy (rows) once, Fx (per column), forms `lam_t = lam_bg + Phi_t .* Fx .* Fy`, calls `pixel_stochastic`, and appends results to one event array that doubles in size when full.
9. Sparse noise	Clean mode only. Adds `noiseRate` x nx x ny x T_sim random events (Poisson count approximated with a normal distribution, so no toolbox is needed).
10. Sort and save	Sorts by time, converts polarity if requested, converts time to integer microseconds, and writes the file with `writematrix`.
11. Ground truth	Prints velocity, angular rate and heading, and a ready-to-paste line of track values for the diagnostics script.
12. Plots	Two quick-look figures: x and y versus time, and the whole-run accumulated event map.

7. How pixel_stochastic works

The local function simulates one pixel photon by photon and returns the event times and polarities.

1. **Start.** V starts at its steady-state value for the light at t = 0 (Eq. 9), so the pixel does not produce a burst of fake events at the start. V_ref is set equal to V.
2. **Photon times.** Because the light changes with time, it uses thinning (Eq. 8). It draws arrivals at the constant maximum rate, then keeps each with probability lambda(t) / lambda_max.
3. **Loop over photons, plus one extra pass for the time after the last photon.** For each pass:
   - Decay crossings: between photons V decays smoothly, so the time it reaches the next threshold can be solved exactly, `t_cross = t_curr + ln(V / target) / a`. If that time is before the next photon, an event is recorded and the check repeats. If V is positive it can only cross the lower threshold; if V is negative it rises toward zero and can cross the upper one.
   - Decay to the next photon time (or the end of the run).
   - The photon arrives and V jumps by h.
   - If the jump crosses a threshold, exactly one event is recorded and V_ref is set to V.
4. **Buffers.** Event arrays double in size when full and are trimmed at the end.

The extra pass after the last photon matters: without it, decay events in the final stretch of the run are lost, which undercounts - events at low flux.

8. Outputs

Event file `synthetic_debris_events.txt`: no header, space-delimited, one event per line, sorted by time.

Column	Content
1	x pixel (1-based, 1 to nx)
2	y pixel (1-based, 1 to ny)
3	polarity, 0/1 or -1/+1 (see `polarity01`)

Column	Content
4	time in integer microseconds from the start of the run

Console: the pixel counts, progress, the total event count split into circuit and noise events, then the ground-truth block and the track line for the diagnostics script.

With the current defaults the ground truth is a pixel speed of about 613.95 px/s, which is about 1025.8 arcsec/s (0.285 deg/s), with a heading of about 176 degrees. In a Python test of the same scene, the circuit produced about 56,000 events and the noise about 193,000, so the file holds roughly 248,000 events.

Figures:

- Figure 1: event density in x versus time and y versus time. The target is a straight line whose slope is its pixel velocity. A dashed line shows the true path.
- Figure 2: every event over the run, drawn on the sensor. The target is a bright streak and the faint speckle is noise.

9. Requirements and performance

- Base MATLAB only; no toolboxes.
- Local functions inside a script need R2016b or newer. `writematrix` needs R2019a or newer. `clim` needs R2022a; the script falls back to `caxis` automatically.
- Clean mode simulates only the box around the track and is fast.
- Full noise mode simulates all 921,600 pixels of a 1280x720 sensor, so expect several minutes. I have not timed this in MATLAB. Crop `nx`, `ny` and shorten `T_sim` while tuning.

10. What has been validated

I could not run MATLAB in my environment. The pixel logic was ported line for line to Python and checked there; the MATLAB file itself has not been executed by me.

- **Single-pixel sweep (paper Fig. 1b, a = b = 1, lambda_r = 0.1, Delta_lam = 0.1):** total rate peaked at about 66 events/s near lambda = 30, with about 28 positive and 39 negative events/s. The paper shows about 65, with 27 positive and 41 negative. The rate fell toward zero at both ends.

- **Background formula:** measured 56.0 events/s/pixel against about 56 predicted (a = 2, Delta_lam = 0.13), and 361 against about 371 predicted (a = 10, Delta_lam = 0.10).
- **Clean mode:** zero background events in 200 pixel-runs of 2 s.
- **Velocity recovery:** on a test scene, positive-event centroids in 50 ms bins recovered (149.98, 19.99) px/s against the true (150, 20).
- **Pipeline check on this simulator's file:** your angular-rate pipeline reported 1024.3 arcsec/s against the true value of about 1025.8, which is about 0.15% low.

11. Assumptions

The model choices and default values below are what the simulator assumes. Values marked unvalidated have not been checked against real event data; the Calibration Plan describes how to do that.

11.1 Qualitative assumptions

Area	Assumption	If it is wrong
Pixel independence	Every pixel is simulated on its own. There is no crosstalk, shared bias noise or spatial correlation of events.	Real data may show clustered or correlated noise that the simulator cannot produce.
Circuit model	The pixel is a first-order leaky integrator of log photon flux: an instant jump per photon and exponential decay between photons (Eqs. 1-4).	A real circuit with a different response changes the tail shape, not only its length.
Photon statistics	Photon arrivals are Poisson, with a time-varying rate where the light changes. The only randomness in the light is photon shot noise.	Seeing flicker and scintillation add variance the model lacks. The paper saw higher central event rates on bright stars than its model predicted.
Thresholds	One global threshold, the same for every pixel. The + and - thresholds both come from Delta_lam, so they are symmetric in relative brightness.	Real pixels have mismatched thresholds and sometimes separate ON and OFF settings, which changes the + and - balance.

Area	Assumption	If it is wrong
Event rule	An event fires when V - V_ref reaches a threshold, then V_ref resets to the current V (paper convention). One large voltage jump makes exactly one event.	A real pixel could emit several events for one large step, so bright onsets are undercounted.
Timing	Events have exact continuous times, rounded to 1 microsecond on export. There is no refractory period, bandwidth limit, readout contention, latency or timestamp jitter.	Real cameras drop or delay events at high rates, which matters near the bright bow shock.
Start-up	Each pixel starts at its steady-state voltage for the light at t = 0.	A real camera may show a start-up burst.
Optics	The target is a point source with a fixed, circular Gaussian blur. There is no seeing variation, diffraction rings, defocus, distortion or vignetting.	A defocused or non-Gaussian blur changes the event footprint; the paper notes out-of-focus stars show donut-shaped event patterns.
Pixel response	Each pixel integrates the blur over its full square area with uniform sensitivity and 100% fill factor. All pixels have identical gain and quantum efficiency.	Real fill factor is below 100% (77% for the EVK3 in the paper) and pixels vary.
Target	One target on a straight line at constant velocity, with steady or sinusoidal brightness. There are no glints, tumbling, acceleration or occultations.	Real targets can change brightness abruptly, as the paper's Ajisai light curves show.
Scene	There are no stars. The sky is spatially uniform and constant in time: no gradients, moon, clouds or other objects.	Real fields have stars that fire events and leave extra streaks.

Area	Assumption	If it is wrong
Brightness units	`Phi_0` is an effective photon flux at the focal plane. It is not computed from magnitude, aperture, quantum efficiency or atmospheric transmission.	It is a tuning knob, so do not read it as a physical magnitude.
Clean mode	The sky sits exactly at the reference flux (lambda_r = lam_bg), so it is invisible by construction. The added noise is independent of light and circuit, uniform in space and time, and 50/50 in polarity.	Real noise is not uniform: it has hot pixels, and its polarity balance may differ.
Full noise mode	All background events come from sky photon shot noise passing through the circuit. There is no dark-current or leakage noise.	Real background noise has other sources, so matching the rate does not match its structure.
Light lookup	The target's light is treated as constant over each `dt` step (nearest sample). Photon times themselves are continuous.	A coarse dt shifts the effective target position by up to half a step (see 11.2).
Coordinates	Pixel (ix, iy) is 1-based, its center is at integer coordinates, and it spans plus or minus 0.5 px. Image y points down.	A 0-based convention in another tool shifts positions by 1 px.
Angle conversion	Small-angle IFOV = 2 atan(pitch / (2 x focal length)) on a flat focal plane.	Distortion would change the angular rate toward the field edges.
Camera processing	No on-camera filtering (hot-pixel, anti-flicker, event-rate control) is applied.	Such filters change real noise and rates, so find out whether they were on for the real data.

11.2 Quantitative assumptions

Quantity	Value	Basis	Confidence
Sensor	1280 x 720 px	Prophesee EVK3 HD, as used in the paper	Known hardware
Pixel pitch, focal length	4.86 um, 0.6 m	EVK3 HD behind an f/3, 0.2 m telescope, from the paper	Known hardware
IFOV	1.671 arcsec/px	2 atan(pitch / (2 x focal length))	Derived
Decay rate `a`	10 1/s (time constant 0.1 s)	Paper Fig. 3 value	Unvalidated. The paper says its circuit values are almost certainly different from the real camera's.
Gain `b`	100	Paper Fig. 3 value	Does not affect event rates
Threshold `Delta_lam`	0.10	Paper Figs. 1-4 value; the EVK3 default is quoted as about 0.25	Check your camera's bias setting
Event thresholds	c_pos = +0.953 V, c_neg = -1.054 V	(b/a) ln(1 +/- Delta_lam), with b/a = 10	Derived
Sky flux `lam_bg`	5 photons/s/pixel	Chosen value	Not derived from sky brightness, aperture or exposure
Target flux `Phi_0`	12,000 photons/s	Chosen value	Unvalidated; effective only
Brightest-pixel rate	about 850 photons/s (about 170 times the sky)	Phi_0 / (2 pi sigma^2) at the centroid	Derived

Quantity	Value	Basis	Confidence
Brightest-pixel voltage	about 51 V (with lambda_r = lam_bg)	(b/a) ln(850 / 5)	Derived
Blur `sigma_psf`	1.5 px (FWHM 3.5 px, about 5.9 arcsec)	Close to the 6 arcsec seeing the paper assumes	Check against real star images
Track	From (1280, 90) at (-612.44, +43.06) px/s for 2.09 s	Chosen to cross the full sensor width	Replace with the real pass
Speed	613.95 px/s = 1025.8 arcsec/s = 0.285 deg/s, heading about 176 degrees	Derived	Derived
Tail length scale	speed / a = about 61 px	Rough v/a scaling	A Python test gave a - event median of -49.7 px and a 10th percentile of -143 px
Time step `dt`	1 ms	Chosen	See the flag below
Motion per step	0.61 px at the default speed	speed x dt	Exceeds the 0.3 px guideline in section 5
Edge margin	m = ceil(5 sigma) + 1 = 9 px	Blur treated as zero beyond 5 sigma	Light beyond 5 sigma is under 1e-6 per axis
Noise rate `noiseRate`	0.10 events/s/pixel (about 192,600 events over 2.09 s)	Chosen value	Replace with the real measured mean rate
Noise count	Mean 192,600, standard deviation about 440	Poisson approximated by a normal distribution	Accurate for means far above 1
Noise polarity and time	50/50, uniform from 0 to T_sim	Assumed	Unvalidated

Quantity	Value	Basis	Confidence
`lambda_r_full`	4.8 photons/s	Gives about 1.1 events/s/pixel at a = 10, Delta_lam = 0.10	Python port only
Event timestamps	Integer microseconds	Matches the EVK3's 1 microsecond resolution	Known
Photon candidates	n = 1.2 lambda_max T + 10 sqrt(lambda_max T) + 20, extended if too few	Implementation detail	No effect on statistics
Decay crossings	At most 2000 between photons	Safety limit; the default needs about 50	No effect at default values
Event buffer	Starts at 1e6 rows and doubles when full	Implementation detail	No effect on results
Random seed	1	Repeatability	Change for new realizations

Flag on the time step. With `dt` = 1 ms and the default speed, the target moves 0.61 px per step, about twice the 0.3 px guideline in section 5. The blur is 1.5 px wide, so the track is still resolved (the pipeline recovered the rate within about 0.15%), but each photon sees the target position of the nearest sample, which can be off by up to about 0.3 px. Setting `dt` to 5e-4 brings motion per step to 0.31 px at the cost of twice as many samples.

12. Limitations

- No refractory period, bandwidth limit, hot pixels or fixed-pattern noise. Real cameras have a heavy tail of busy pixels that this model does not produce.
- Clean-mode noise is uniform and exactly 50/50 in polarity, and is not derived from circuit physics.
- One target, constant velocity, fixed Gaussian blur. There is no seeing flicker and no non-Gaussian blur.
- A single large voltage jump produces one event, as in the paper, where a real pixel might produce several.

- The target's light is sampled on the `dt` grid, so very fast targets need a smaller dt.
- The circuit is a first-order model; the real circuit's behavior may differ, and the paper notes the true circuit parameters are not known.
- Setting lambda_r equal to lam_bg (clean mode) is a modeling shortcut, not a physical sky.

13. Troubleshooting

Symptom	Likely cause and fix
Warning: track leaves the sensor or passes within 9 px of an edge	The blur is clipped at the edge. The default track starts at x = 1280 and ends near x = 0, so this appears. Move the track inward if you want an unbiased fit.
Error: full-noise mode would create about N events	The estimate exceeded `maxEventsAllowed`. Reduce nx, ny or T_sim, raise lambda_r_full, or raise the cap.
Pipeline finds nothing	In full noise mode with a low lambda_r the background can make every pixel look steady and a steady-source mask removes everything. Use clean mode or raise lambda_r_full.
`writematrix` not found	MATLAB older than R2019a. Use `dlmwrite` with a space delimiter.
Almost no events	Check that the track is on the sensor and that Phi_0 and Delta_lam are reasonable.
Run takes very long	Full noise mode on a full sensor. Crop the sensor or shorten T_sim.

14. Change history

- **Original upload:** one script with deterministic and stochastic modes and a full-sensor loop. Decay events after the last photon were never generated.

- **v2:** added the post-last-photon pass, handling of negative voltage jumps, a track-window option, doubling buffers, a clean background mode, and a single-pixel validation sweep.
- **v3:** removed the deterministic mode and the validation function, renamed the paper-faithful mode to 'full noise', added extensive comments, and moved the calibration plots to Event_Data_Diagnostics.m.